Rules & Carbon Engine — Build Specification

Project: Glass-Box Green Insights (Lloyds Megathon, Problem Statement 1) **Owner:** Person 3 — Rules & Carbon Engine **Audience:** AI coding agent (Antigravity). Build exactly what is specified. Where something is marked `TO_VERIFY`, do not invent a value — leave it in config and flag it.

Key terms (from the Lloyds problem statement — use these exact meanings in code, docs and UI)

Term	Meaning in this module
Cutoff date	The last date covered by the input data. Every feature must be dated on or before it. Stored as `cutoff_date` and echoed in every output.
Take-up / outcome window	A customer "takes up" a product if they adopt it within a fixed window after the cutoff date. The team must state the window (`outcome_window_days`, default 90). Owned by Person 1/2; this module only carries it through.
Propensity	Estimated probability of take-up, in [0, 1]. Produced by ML; passed through unchanged.
Confidence	How reliable the system considers a propensity estimate, in [0, 1]. Method is the team's choice and documented by Person 2. **Distinct from propensity and never an indicator of financial eligibility.**
Reason code	A fixed identifier for one factor behind a recommendation (e.g. `HIGH_FUEL_SPEND`). Chosen from a fixed registry; this module never invents one.
Carbon engine	Deterministic, rules-based component that calculates estimated emissions or emissions avoided from activity data, documented formulas/baselines and emissions factors. **It makes no behavioural predictions and does not determine eligibility.**
Financial eligibility	Whether a customer qualifies under the institution's **authorised product criteria and processes**. Propensity is not eligibility.
Estimate	A modelled potential value, not a measured result. **All impact figures are estimates** and must be labelled as such everywhere.

---

0. Golden rules (non-negotiable)

1. **ML predicts. Rules decide. Carbon engine calculates. LLM explains.** This module is rules + carbon only. It never calls an ML model or an LLM.
2. A high propensity score can **never** override a failed eligibility rule.
3. Every calculation is **deterministic pure Python**: same input + same factor/rule version = same output. No randomness, no network calls, no LLM calls.
4. **Never invent a missing value.** Missing required input → `insufficient_data`, never a guess.
5. Every impact is a **range** (low / central / high) with unit, period, method, assumptions, factor IDs and versions.
6. **CO2e, energy (kWh), paper (sheets/kg) and digital adoption are reported separately.** Never merge them.
7. Impact is **never multiplied by propensity**. "Potential impact if the customer adopts" ≠ "expected impact".

8. Thresholds, factors and assumptions live in **config files** (YAML/CSV), never hard-coded in calculator logic.
9. **Financial eligibility** is decided only by rules that implement the product's **authorised criteria** (from the green-products reference catalogue). Propensity and confidence never affect it. Customer-facing wording must stay "subject to eligibility checks"; final approval remains with the institution's normal processes.
10. **Every impact is an estimate.** Output carries `is_estimate: true` and wording uses "estimated" / "potential". Never present it as a measured or guaranteed result.
11. **Cutoff discipline.** No input dated after `cutoff_date` may be used (prevents leakage).

---

1. Pipeline position

- Step 2 runs only if eligibility ∈ {`ELIGIBLE`, `ELIGIBLE_WITH_CONDITIONS`}. Otherwise impact = `not_applicable` with reason `BLOCKED_BY_ELIGIBILITY` (or `INSUFFICIENT_DATA` if that was the eligibility result).
- Steps 1 and 2 must be callable independently (for unit tests and the "what-if" demo).

---

2. Repository layout

Adapt to the team repo; do not duplicate existing app files.

```
carbon_engine/
  __init__.py
  schemas.py          # pydantic models: inputs, EligibilityResult, ImpactResult,
Opportunity
  units.py            # explicit unit conversion + rejection of unknown units
  factors.py          # load/validate factor + assumption registers, version selection
  ranges.py           # Range dataclass + interval arithmetic helpers
  ev.py  solar.py  green_home.py  paperless.py
eligibility/
  __init__.py
  dsl.py              # rule condition evaluator
  engine.py           # evaluate_eligibility(customer_ctx, product) ->
EligibilityResult
recommender/
  gate.py             # propensity/confidence gate
  priority.py         # priority score + per-customer ranking
  orchestrator.py     # evaluate_opportunity(...) -> Opportunity
config/
  rules_v1.yaml
  thresholds.yaml
  green_home_interventions.yaml
  reason_codes.yaml     # fixed reason-code registry (owned by Person 2; read-only
here)
data/
  emissions_factors.csv
  assumptions.csv
app/api/carbon.py     # thin FastAPI wrapper (built LAST)
tests/                # test_ev.py test_solar.py test_green_home.py test_paperless.py
                      # test_eligibility.py test_gate_priority.py test_factors.py
test_units.py
docs/carbon_methodology.md
```

Calculators must not import FastAPI, Streamlit or any LLM SDK.

---

3. Input contract (agree field names with Person 1 — Data)

The orchestrator receives a flat `customer_ctx` dict built from the Customer 360 table. **These are proposed names; if Person 1 uses different names, add a single mapping layer, do not change the rules.**

Field	Type	Notes
`customer_id`	str	
`cutoff_date`	date	last date covered by the input data; all features dated ⩽ this
`outcome_window_days`	int	take-up window after the cutoff (team-stated, default 90); passed through
`country`	str	`GB` expected (UK factors)
`age_band`	str	e.g. `<18`, `18-24`, …
`account_status`	str	`ACTIVE` / other
`green_offers_opt_out`	bool	marketing preference
`vulnerability_flag`	bool	
`has_arrears`	bool	
`affordability_prescreen`	str	`PASS` / `REFER` / `FAIL` (synthetic)
`held_products`	list[str]	product IDs already held
`home_ownership`	str	`OWNER_OUTRIGHT` / `MORTGAGE` / `RENTER` / `OTHER`
`property_type`	str?	`HOUSE` / `FLAT` / … optional
`heating_type`	str?	`GAS_BOILER` / `HEAT_PUMP` / `ELECTRIC` / … optional
`insulation_status`	str?	`NONE` / `PARTIAL` / `FULL` optional
`has_solar`	bool?	
`vehicle_fuel_type`	str?	`PETROL` / `DIESEL` / `HYBRID` / `PHEV` / `BEV`
`vehicle_age_years`	float?	
`vehicle_finance_months_remaining`	float?	
`annual_distance` + `annual_distance_unit` + `annual_distance_source`	num/str/str	source ∈ `ODOMETER`, `DECLARED`, `SPEND_DERIVED`
`annual_fuel_spend_gbp`	float?	from categorised transactions
`electricity_kwh_monthly`	list[12] of float|null	most recent 12 months
`gas_kwh_monthly`	list[12] of float|null	
`months_electricity_data` / `months_gas_data`	int	non-null months

Field	Type	Notes
`paper_statements`	bool?	
`statements_per_year`	int?	
`pages_per_statement`	float?	optional measured value
`digital_logins_90d`	int?	app+web
`requires_paper_comms`	bool?	accessibility / customer need

Null/absent field = missing. The engine must distinguish `False` from missing.

---

4. Financial eligibility engine

Rules implement the **authorised product criteria** from the green-products reference catalogue (`criteria_source: product_catalogue_v1`). Each rule records which catalogue criterion it implements, so the audit view can show "blocked because criterion X of product Y failed".

4.1 Result statuses (precedence, highest first)

`NOT_ELIGIBLE` > `INSUFFICIENT_DATA` > `ELIGIBLE_WITH_CONDITIONS` > `ELIGIBLE`

Evaluate ALL applicable rules (no short-circuit) so the audit view can show every pass/fail/missing.

4.2 Rule severities

Severity	Fail effect
`HARD_BLOCK`	status → `NOT_ELIGIBLE`
`REQUIRES_DATA`	missing field → `INSUFFICIENT_DATA`
`CONDITION`	adds a condition string; status → `ELIGIBLE_WITH_CONDITIONS`

Each rule has `on_missing`: `MISSING` (→ INSUFFICIENT_DATA) or `PASS`. Default `MISSING` (conservative).

4.3 Rule DSL

yaml

```yaml
- id: EV01
  products: [EV_FINANCE]
  severity: HARD_BLOCK
  description: Existing BEV owner is not offered EV Finance unless a replacement case ex
  when_fails_if:           # rule FAILS when this block is true
    all:
      - {field: vehicle_fuel_type, op: eq, value: BEV}
      - not:
          any:
            - {field: vehicle_age_years, op: gte, value: "$thr.ev_replacement_min_age_ye
            - {field: vehicle_finance_months_remaining, op: lte, value: "$thr.ev_replacer
  on_missing: PASS
  fail_code: EL_FAIL_EXISTING_EV_NO_REPLACEMENT
  pass_code: EL_PASS_EV_OWNERSHIP
```

Supported ops: `eq ne in not_in gt gte lt lte is_true is_false is_null not_null contains not_contains`. Combinators: `all`, `any`, `not`. Values starting with `$thr.` resolve from `config/thresholds.yaml`. **No `eval()`** — the evaluator is a small explicit interpreter.

4.4 Thresholds (`config/thresholds.yaml`) — team assumptions, versioned

yaml

```yaml
rule_version: rules_v1.0.0
data_max_age_days: 90   # measured as (run date − cutoff_date)
outcome_window_days: 90 # take-up window after cutoff; must match Person 1/2's label def
ev_replacement_min_age_years: 8
ev_replacement_max_finance_months: 6
min_electricity_kwh_year_solar: 1500
min_months_electricity_solar: 6
min_months_energy_green_home: 12
min_digital_logins_90d: 6
```

The problem statement leaves the catalogue criteria to the team's synthetic reference data, so these thresholds **are** the authorised criteria for this build: record them in the product catalogue, version them (`rule_version`), and state in the methodology doc and Responsible-AI note that they are synthetic and not real Lloyds criteria.

4.5 Rules

Global (all four products)

ID	Severity	Fails when	Code
G01	HARD_BLOCK	`age_band` = `<18`	`EL_FAIL_UNDER_18`
G02	HARD_BLOCK	`green_offers_opt_out` is true (missing → MISSING)	`EL_FAIL_OPTED_OUT`
G03	HARD_BLOCK	`account_status` ≠ `ACTIVE`	`EL_FAIL_ACCOUNT_INACTIVE`
G04	REQUIRES_DATA	run date − `cutoff_date` > `data_max_age_days`	`EL_MISSING_STALE_DATA`
G05	HARD_BLOCK	product already in `held_products`	`EL_FAIL_ALREADY_HOLDS_PRODUCT`
G06	CONDITION	`vulnerability_flag` true → "Route to human review; no automated customer message"	`EL_COND_HUMAN_REVIEW`

Lending products only (EV_FINANCE, GREEN_HOME, SOLAR_FINANCE)

ID	Severity	Fails when	Code
L01	HARD_BLOCK	`has_arrears` true	`EL_FAIL_ARREARS`
L02	HARD_BLOCK	`affordability_prescreen` = `FAIL`	`EL_FAIL_AFFORDABILITY_PRESCREEN`
L03	CONDITION	`affordability_prescreen` = `REFER`	`EL_COND_AFFORDABILITY_REFER`
L04	CONDITION (always)	"Subject to eligibility and credit checks"	`EL_COND_SUBJECT_TO_CHECKS`

EV_FINANCE

ID	Severity	Rule	Code
EV01	HARD_BLOCK	BEV owner **unless** `vehicle_age_years ⩾ 8` OR `vehicle_finance_months_remaining ⩽ 6`	`EL_FAIL_EXISTING_EV_NO_REPLACEMENT`
EV01b	CONDITION	BEV owner who *does* pass via replacement case	`EL_COND_EV_REPLACEMENT_CASE`
EV02	REQUIRES_DATA	needs a baseline: `vehicle_fuel_type` ∈ {PETROL, DIESEL, HYBRID, PHEV} **or** `annual_fuel_spend_gbp` > 0. Neither → missing	`EL_MISSING_VEHICLE_BASELINE`
EV03	CONDITION	No vehicle record but fuel spend > 0 → "Vehicle type unverified"	`EL_COND_VEHICLE_UNVERIFIED`
EV04	CONDITION	PHEV owner (upgrade to full BEV)	`EL_COND_PHEV_UPGRADE`

GREEN_HOME — *Scope definition (hard boundary):* covers **home fabric and heating efficiency only** (loft insulation, wall insulation, heat pump). It **excludes rooftop solar PV, batteries and EV chargers**, which belong to Solar Finance / EV Finance. Document this in the catalogue and methodology.

ID	Severity	Rule	Code
GH01	HARD_BLOCK	`home_ownership` ∈ {OWNER_OUTRIGHT, MORTGAGE} else fail	`EL_FAIL_NOT_HOMEOWNER`
GH02	REQUIRES_DATA	`months_electricity_data` and `months_gas_data` ⩾ 12 (at least one fuel with ⩾ 12 months)	`EL_MISSING_ENERGY_HISTORY`
GH03	HARD_BLOCK	No applicable intervention in catalogue (e.g. already `HEAT_PUMP` and `insulation_status` = `FULL`)	`EL_FAIL_NO_APPLICABLE_INTERVENTION`

SOLAR_FINANCE

ID	Severity	Rule	Code
SO01	HARD_BLOCK	same homeowner rule as GH01	`EL_FAIL_NOT_HOMEOWNER`
SO02	HARD_BLOCK	`has_solar` true. **Missing → MISSING** (absence of solar must be evidenced)	`EL_FAIL_EXISTING_SOLAR`
SO03	HARD_BLOCK	`property_type` = `FLAT`	`EL_FAIL_ROOF_UNSUITABLE`
SO03b	CONDITION (always)	"Subject to roof suitability survey"	`EL_COND_ROOF_SURVEY`
SO04	REQUIRES_DATA	`months_electricity_data` ⩾ 6	`EL_MISSING_ELECTRICITY_HISTORY`
SO05	HARD_BLOCK	annualised electricity < `min_electricity_kwh_year_solar`	`EL_FAIL_LOW_ELECTRICITY_USE`

PAPERLESS_BANKING (no lending rules)

ID	Severity	Rule	Code
PB01	HARD_BLOCK	`paper_statements` is false (already paperless). Missing → MISSING	`EL_FAIL_ALREADY_PAPERLESS`
PB02	HARD_BLOCK	`digital_logins_90d` < `min_digital_logins_90d`	`EL_FAIL_LOW_DIGITAL_ACTIVITY`
PB03	HARD_BLOCK	`requires_paper_comms` is true (accessibility/customer need)	`EL_FAIL_PAPER_NEEDED`

4.6 Eligibility output

json


```
{
  "customer_id": "C10001",
  "product": "EV_FINANCE",
  "eligibility_status": "NOT_ELIGIBLE",
  "conditions": [],
  "decisions": [
    {"rule_id": "EV01", "outcome": "FAIL", "code": "EL_FAIL_EXISTING_EV_NO_REPLACEMENT",
     "severity": "HARD_BLOCK", "criteria_source": "product_catalogue_v1:EV_FINANCE:no_ex
     "evidence": {"vehicle_fuel_type": "BEV", "vehicle_age_years": 2.1},
     "rule_version": "rules_v1.0.0"},
    {"rule_id": "G01", "outcome": "PASS", "code": "EL_PASS_ADULT", "severity": "HARD_BLO
     "evidence": {"age_band": "35-44"}, "rule_version": "rules_v1.0.0"}
  ],
  "evaluated_at": "2026-10-10T00:00:00Z",
  "rule_version": "rules_v1.0.0"
}
```


`evidence` holds only the fields the rule used. Never include free text from the customer.

---

5. Recommendation gate and priority (after eligibility + carbon)

5.1 Gate (`recommender/gate.py`)

Input validation first: `propensity` and `confidence` must each be numbers in [0, 1] and every `reason_code` must exist in `config/reason_codes.yaml`; otherwise reject with HTTP 422 / `INVALID_ML_PAYLOAD`. This module never edits or invents them. **Confidence is never used as an eligibility input.**

A product reaches the customer/RM view only if **all** are true:

1. `eligibility_status` ∈ { `ELIGIBLE`, `ELIGIBLE_WITH_CONDITIONS` }
2. `propensity ≥ MIN_PROPENSITY` (default 0.50, per product override allowed in `thresholds.yaml`; Person 2 tunes using precision@k)
3. `confidence ≥ MIN_CONFIDENCE` (default 0.50)
4. impact is not a claimed reduction that is ≤ 0: if `impact.co2e.central ≤ 0` and `claim_reduction=false`, the product is **not** presented as a carbon saving (it may still be shown only if the product's value is non-carbon, i.e. Paperless).

Gate failures produce codes: `GATE_LOW_PROPENSITY`, `GATE_LOW_CONFIDENCE`, `GATE_NO_REDUCTION`, `GATE_BLOCKED_ELIGIBILITY`.

5.2 Priority score

```
priority = propensity × confidence × impact_norm
impact_norm = clamp(impact_central / P95(impact_central across eligible customers of
the SAME product), 0, 1)
impact_norm = IMPACT_NORM_FLOOR (default 0.10) when impact status = insufficient_data
```

- Normalise **within product** (CO2e for EV/Solar/GH; **sheets avoided** for Paperless). Never normalise paper against CO2e.
- Always return `propensity`, `confidence`, `impact_norm` **alongside** `priority` (no black box).
- Per customer: rank eligible opportunities by priority; `rank_within_customer = 1` is the

primary recommendation; others are secondary. Green Home and Solar can both be eligible; scope is exclusive by definition, ranking decides which is primary.

---

6. Carbon engine

6.1 Common rules

- **Range type:** `Range(low, central, high)`. Interval arithmetic for reductions: `reduction.low = baseline.low − scenario.high`, `reduction.high = baseline.high − scenario.low`, `reduction.central = baseline.central − scenario.central`. (Conservative envelope; document it.)
- **Signed results:** never clamp to zero. If `central < 0` → `claim_reduction=false` and add limitation `SCENARIO_NOT_LOWER_THAN_BASELINE`. If only `low < 0` → add flag `RANGE_INCLUDES_NO_REDUCTION`.
- **Zero activity:** zero distance / zero statements returns a valid `0` result (status `estimated`), not an error.
- **Cutoff check:** every dated input (transactions, energy months, logins, vehicle records) must be ⩽ `cutoff_date`. Any later value → `calculation_error` `INPUT_AFTER_CUTOFF`; never silently drop or use it.
- **Estimate labelling:** every impact result has `is_estimate: true` and `estimate_label: "Estimated potential impact, not a measured result"`. Calculators must not use the words "saved" or "will save" in any string they emit.
- **Statuses:** `estimated`, `insufficient_data`, `not_applicable`, `calculation_error`. UI must never show a number unless status = `estimated`.
- **Provenance grade** (on every result):
  - `REPORTABLE` — every factor `VERIFIED_OFFICIAL` and every boundary matched and confirmed
  - `PROVISIONAL` — any factor `AGGREGATOR_UNVERIFIED` or `DEV_ILLUSTRATIVE`, or boundary `UNCONFIRMED`
  - UI shows an "illustrative" badge for `PROVISIONAL`. Final submission numbers must be `REPORTABLE`.
- **Units:** accept only units in `units.py` (`km`, `mile`, `kWh`, `Wh`, `MWh`, `L`, `kg`, `g`, `sheet`). Convert explicitly (`mile→km ×1.609344`, `Wh→kWh ÷1000`). Unknown unit → `calculation_error` with `UNSUPPORTED_UNIT`. Never silently mix units.
- **Geography/period:** factor geography must equal `country` (`GB`) and factor year must match activity year (or be flagged). Mismatch → `PROVISIONAL` + limitation, or `not_applicable` if geography is wrong.
- **Boundary consistency (EV):** baseline and EV scenario must use the **same boundary** string; mismatch → `calculation_error` `BOUNDARY_MISMATCH` (`UNCONFIRMED` on both is allowed but forces `PROVISIONAL`).
- **Minimum-input quality → range width** (config `thresholds.yaml → input_quality`):

Quality	Meaning	Range widening
`ODOMETER`	measured mileage	±5%
`DECLARED`	customer-declared	±10%
`SPEND_DERIVED`	derived from fuel spend	±25%
`ANNUALISED_PARTIAL`	6–11 months energy annualised	±15%
`FULL_12M`	12 months energy	±0%

6.2 EV Finance (`ev.py`)

```
distance_km  = annual_distance converted to km  (Range via input_quality)
Baseline     = distance_km × ICE_factor[fuel_type]                  (kg CO2e/km)
Scenario     = distance_km × ev_kwh_per_km × GRID_ELEC_UK           (kWh/km × kg
CO2e/kWh)
Reduction    = Baseline − Scenario                                  (kg CO2e/year)
```

- `ICE_factor` chosen by `vehicle_fuel_type` (PETROL/DIESEL/HYBRID; PHEV uses `CAR_PHEV_AVG_KM` factor if present, else `insufficient_data`).
- `ev_kwh_per_km` is a Range from `assumptions.csv`.
- **Missing distance:** if `annual_distance` missing and `ALLOW_SPEND_DERIVED_DISTANCE=true` and `annual_fuel_spend_gbp` present → `distance_km = spend / FUEL_PRICE_PER_LITRE × ICE_KM_PER_LITRE` with quality `SPEND_DERIVED`, plus limitation `DISTANCE_DERIVED_FROM_SPEND`. Otherwise → `insufficient_data` (`IMPACT_MISSING_ANNUAL_DISTANCE`).
- Also report `energy`: not required for EV (leave `null`).
- Method name: `EV_ANNUAL_DISTANCE_X_FACTOR_v1`.

6.3 Solar Finance (`solar.py`)

```
annual_kwh          = Σ electricity_kwh_monthly (annualised if 6–11 months: mean × 12)
system_kwp          = clamp(round(annual_kwh / 1000, 1), SOLAR_SIZE_MIN, SOLAR_SIZE_MAX)
# sizing rule v1
generation_kwh      = system_kwp × SOLAR_YIELD                          (Range)
displaced_kwh       = min(generation_kwh × SOLAR_SELF_CONSUMPTION, annual_kwh)   (Range)
CO2e avoided        = displaced_kwh × GRID_ELEC_UK
```

- **Only self-consumed generation counts.** Exported electricity is reported as `exported_kwh` (informational) but **not** counted in CO2e.
- Report `energy.displaced_kwh` (Range) separately from `co2e`.
- Cap: displaced can never exceed the customer's annual grid use.
- Method name: `SOLAR_SELF_CONSUMED_X_GRID_FACTOR_v1`.

6.4 Green Home (`green_home.py`)

Interventions come from `config/green_home_interventions.yaml` (catalogue). **No generic improvement percentage without a recorded source.**

```
baseload_gas   = mean(Jun, Jul, Aug monthly gas kWh) × 12
heating_gas    = max(0, annual_gas_kWh − baseload_gas)          # space-heating
share, derived from the customer's own data
```

Intervention selection (deterministic, first match):

1. `heating_type = GAS_BOILER` and `heating_gas ≥ 3000 kWh` → `HEAT_PUMP`
2. `insulation_status ∈ {NONE, PARTIAL}` or unknown, `heating_gas ≥ 1500` → `LOFT_AND_WALL_INSULATION`
3. else → no applicable intervention (feeds rule GH03)

Scenario formulas

- `HEAT_PUMP`:

```
  heat_delivered = heating_gas × BOILER_EFFICIENCY
  hp_electricity = heat_delivered / HP_SCOP
  Baseline  = annual_gas × GAS_FACTOR + annual_elec × GRID
  Scenario  = (annual_gas − heating_gas) × GAS_FACTOR + (annual_elec + hp_electricity)
× GRID
```

(Hot water and cooking assumed to stay on gas — documented simplification.)

- `LOFT_AND_WALL_INSULATION`:

```
  Scenario gas = annual_gas − heating_gas × HEATING_REDUCTION_FRACTION    (Range from
assumptions)
```

- Report `energy` separately (baseline kWh, scenario kWh, change), and CO2e separately.
- Pair worst/best-case assumption values per the interval rule in 6.1 (`BOILER_EFFICIENCY` high + `HP_SCOP` low = worst case).
- Method names: `GH_HEAT_PUMP_SEASONAL_SPLIT_v1`, `GH_INSULATION_FRACTION_v1`.

6.5 Paperless Banking (`paperless.py`)

```
statements_avoided = statements_per_year                          (default 12 if monthly
flag; else insufficient_data)
sheets_avoided     = statements_avoided × SHEETS_PER_STATEMENT (Range 1/2/3 unless
pages_per_statement measured)
paper_kg_avoided   = sheets_avoided × SHEET_MASS_KG
```

- Primary outputs: `statements_avoided`, `sheets_avoided`, `paper_kg_avoided`. These are the **paper** metrics.
- **CO2e is NOT reported by default** (`ENABLE_PAPER_CO2E=false`). Only enable when a `VERIFIED_OFFICIAL` paper factor and a documented net-of-digital-footprint method exist. If disabled: `co2e = {status: "not_reported", reason: "NO_DOCUMENTED_METHOD"}`.
- **Digital adoption is never described as a carbon saving.** Separate field `digital_adoption: {statements_moved_to_digital: n}`.
- Method name: `PAPER_STATEMENTS_TO_DIGITAL_v1`.

6.6 Insufficient-data codes

`IMPACT_MISSING_ANNUAL_DISTANCE`, `IMPACT_MISSING_VEHICLE_FUEL_TYPE`,

`IMPACT_MISSING_ELECTRICITY_USAGE`, `IMPACT_MISSING_GAS_USAGE`, `IMPACT_INSUFFICIENT_MONTHS`, `IMPACT_MISSING_STATEMENT_COUNT`, `IMPACT_MISSING_FACTOR`, `IMPACT_FACTOR_NOT_REPORTABLE`.

---

7. Registers (data files)

7.1 `data/emissions_factors.csv`

Columns: `factor_id, activity, factor_value, factor_unit, activity_unit, source_name, source_url, geography, period, version, boundary, verification_status, notes`

`verification_status` ∈ `VERIFIED_OFFICIAL`, `AGGREGATOR_UNVERIFIED`, `DEV_ILLUSTRATIVE`.

Seed rows (the owner must replace/verify against the official DESNZ 2026 "flat file" before final reporting):

csv

```
factor_id,activity,factor_value,factor_unit,activity_unit,source_name,source_url,geograph
GRID_ELEC_UK,Grid electricity consumed,0.15,kg_CO2e/kWh,kWh,PLACEHOLDER - replace with DE
GAS_NATURAL_GCV,Natural gas (gross CV),0.18231,kg_CO2e/kWh,kWh,DESNZ 2026 via third-party
CAR_PETROL_AVG_KM,Average petrol car distance,0.17,kg_CO2e/km,km,PLACEHOLDER,https://www
CAR_DIESEL_AVG_KM,Average diesel car distance,0.17,kg_CO2e/km,km,PLACEHOLDER,https://www
CAR_HYBRID_AVG_KM,Average hybrid car distance,0.12,kg_CO2e/km,km,PLACEHOLDER,https://www
EV_BEV_AVG_KM_XCHECK,BEV average car charging (cross-check only),0.02686,kg_CO2e/km,km,DE
```

Notes for the loader:

- Reject rows with missing `factor_value`, `factor_unit`, `source_name`, or `geography`.
- Select factors by `factor_id` + `period` matching the activity year; multiple versions allowed, newest `version` for that year wins unless the caller pins one.
- Expose `factor_version` = a stable hash/ID of the register snapshot used for the run.

7.2 `data/assumptions.csv`

Columns: `assumption_id, low, central, high, unit, source_name, status, notes`. All rows below start as `DEV_ILLUSTRATIVE` — **each must get a cited source before final reporting** (e.g. Energy Saving Trust, DESNZ, DfT, MCS).

csv

```
assumption_id,low,central,high,unit,source_name,status,notes
EV_KWH_PER_KM,0.17,0.19,0.22,kWh/km,TO_CITE,DEV_ILLUSTRATIVE,"Cross-check against DESNZ S
FUEL_PRICE_PER_LITRE,1.30,1.45,1.60,GBP/L,TO_CITE,DEV_ILLUSTRATIVE,"Only for spend-derive
ICE_KM_PER_LITRE,11,13,15,km/L,TO_CITE,DEV_ILLUSTRATIVE,"Only for spend-derived distance"
SOLAR_YIELD,750,850,950,kWh/kWp/yr,TO_CITE,DEV_ILLUSTRATIVE,"UK typical range"
SOLAR_SELF_CONSUMPTION,0.30,0.40,0.50,fraction,TO_CITE,DEV_ILLUSTRATIVE,"No battery"
SOLAR_SIZE_MIN,2,2,2,kWp,TO_CITE,DEV_ILLUSTRATIVE,"Sizing clamp"
SOLAR_SIZE_MAX,6,6,6,kWp,TO_CITE,DEV_ILLUSTRATIVE,"Sizing clamp"
BOILER_EFFICIENCY,0.80,0.88,0.92,fraction,TO_CITE,DEV_ILLUSTRATIVE,""
HP_SCOP,2.5,3.0,3.5,ratio,TO_CITE,DEV_ILLUSTRATIVE,""
HEATING_REDUCTION_LOFT_WALL,0.10,0.20,0.30,fraction,TO_CITE,DEV_ILLUSTRATIVE,"Fraction of
SHEETS_PER_STATEMENT,1,2,3,sheet,TO_CITE,DEV_ILLUSTRATIVE,"Use measured pages if availabl
SHEET_MASS_KG,0.00499,0.00499,0.00499,kg/sheet,Calculated: A4 210x297mm x 80gsm,DERIVED,"
```

---

8. Output contracts

8.1 Impact result (extends the roadmap's contract with ranges + provenance)

json


```
{
  "customer_id": "C10001",
  "product": "EV_FINANCE",
  "status": "estimated",
  "is_estimate": true,
  "estimate_label": "Estimated potential impact, not a measured result",
  "claim_reduction": true,
  "impact_type": "annual_emissions_reduction",
  "co2e": {"low": 1508.4, "central": 1800.0, "high": 2084.4, "unit": "kg_CO2e/year"},
  "energy": null,
  "paper": null,
  "digital_adoption": null,
  "baseline": {"low": 1944.0, "central": 2160.0, "high": 2376.0, "unit": "kg_CO2e/year"}
  "scenario": {"low": 291.6, "central": 360.0, "high": 435.6, "unit": "kg_CO2e/year"},
  "method": "EV_ANNUAL_DISTANCE_X_FACTOR_v1",
  "input_quality": "DECLARED",
  "factor_ids": ["CAR_PETROL_AVG_KM", "GRID_ELEC_UK"],
  "factor_version": "ef-register-2026-xx",
  "assumption_ids": ["EV_KWH_PER_KM"],
  "provenance_grade": "PROVISIONAL",
  "assumptions": ["Annual distance unchanged after switching", "EV efficiency 0.17–0.22 k
  "limitations": ["Estimate, not a measured outcome", "Illustrative factors"],
  "reason_codes": [],
  "cutoff_date": "2026-09-30",
  "calculated_at": "2026-10-10T00:00:00Z",
  "engine_version": "carbon_engine_0.1.0"
}
```


For `insufficient_data`: all value blocks are `null` and `reason_codes` lists the missing-input codes. No numbers anywhere.

8.2 Opportunity (shared format — must match the team JSON, Section 10 of the guide)

json

```
{
  "customer_id": "C10001",
  "product": "EV_FINANCE",
  "propensity": 0.84,
  "confidence": 0.71,
  "reason_codes": ["HIGH_FUEL_SPEND", "HIGH_MILEAGE"],
  "eligibility_result": "ELIGIBLE_WITH_CONDITIONS",
  "eligibility_conditions": ["Subject to eligibility and credit checks"],
  "decision_codes": ["G01:PASS", "EV01:PASS", "L04:CONDITION"],
  "gate": {"passed": true, "codes": []},
  "impact": { "...": "ImpactResult from 8.1" },
  "priority": {"score": 0.43, "impact_norm": 0.72, "rank_within_customer": 1},
  "cutoff_date": "2026-09-30",
  "outcome_window_days": 90,
  "data_timestamp": "2026-09-30",
  "model_version": "<from ML>",
  "rule_version": "rules_v1.0.0",
  "factor_version": "ef-register-2026-xx"
}
```


`propensity` (estimated probability of take-up, [0,1]), `confidence` (reliability of that estimate, [0,1]; not eligibility), `reason_codes`, `model_version`, `cutoff_date` and `outcome_window_days` are **passed through unchanged** from Persons 1/2. This module never modifies them.

---

9. API (build last, thin wrapper)

- `POST /eligibility/evaluate` → `EligibilityResult`
- `POST /impact/calculate` → `ImpactResult` (also used by the what-if calculator: accepts overrides such as `annual_distance`, `system_kwp`; response includes `"scenario": true`)
- `POST /opportunities/evaluate` → `Opportunity` (accepts `customer_ctx` + ML payload)
- `GET /rules` → active rules + thresholds + `rule_version`
- `GET /factors` → factor and assumption registers
- Malformed input / unsupported product → HTTP 422 with a clear error. **Never fabricate a result.**
- Provide a **mock mode** returning clearly labelled mock outputs (`"mock": true`) so UI/LLM teammates can build in parallel.

---

10. Tests (pytest, fixed seeds, all must pass)

10.1 Hand-calculated carbon cases (use `DEV_ILLUSTRATIVE` factors: grid 0.15, gas 0.183, petrol 0.18/km; tolerance ±0.01)

EV — 12,000 km DECLARED (±10% → 10,800 / 12,000 / 13,200), ICE factor 0.18 kg/km, EV efficiency 0.18 / 0.20 / 0.22 kWh/km (test-only values, not the `assumptions.csv` defaults):

- Baseline 1,944 / 2,160 / 2,376
- Scenario (low = 10,800 × 0.18 × 0.15 = 291.6; central = 12,000 × 0.20 × 0.15 = 360; high = 13,200 × 0.22 × 0.15 = 435.6)
- Reduction: low 1,944 − 435.6 = **1,508.4**, central **1,800.0**, high 2,376 − 291.6 = **2,084.4**

Solar — annual electricity 3,600 kWh, FULL_12M. `system_kwp = 3.6`. Yield 750/850/950, self-consumption 0.30/0.40/0.50, grid 0.15:

- Generation 2,700 / 3,060 / 3,420 kWh

- Displaced 810 / 1,224 / 1,710 kWh
- CO2e avoided **121.5** / **183.6** / **256.5** kg/year

Green Home (heat pump) — gas 12,000 kWh/yr, Jun–Aug avg 300 kWh/month, electricity not changed otherwise (baseline electricity ignored in the difference):

- Baseload 3,600; heating gas 8,400
- Central: electricity added 8,400 × 0.88 / 3.0 = 2,464 kWh; reduction = 8,400 × 0.183 − 2,464 × 0.15 = 1,537.2 − 369.6 = **1,167.6** kg
- Worst (eff 0.92, SCOP 2.5): added 3,091.2 kWh → reduction **1,073.52**
- Best (eff 0.80, SCOP 3.5): added 1,920 kWh → reduction **1,249.2**

Paperless — 12 statements/yr:

- Sheets avoided 12 / 24 / 36; paper 0.05988 / 0.11976 / 0.17964 kg; `co2e.status = "not_reported"`

10.2 Eligibility / edge-case tests (required)

1. **High propensity (0.95) but eligibility fails** → never appears in the RM queue; audit shows failing rule.
2. **Existing BEV owner, 2 yrs old, no finance ending** → `NOT_ELIGIBLE` (`EL_FAIL_EXISTING_EV_NO_REPLACEMENT`). Same owner, 9 yrs old → `ELIGIBLE_WITH_CONDITIONS` with `EL_COND_EV_REPLACEMENT_CASE`.
3. **Missing mileage and no fuel spend** → impact `insufficient_data`; no number.
4. **Missing mileage with fuel spend** → spend-derived estimate, quality `SPEND_DERIVED`, ±25% range, limitation present; same input with `ALLOW_SPEND_DERIVED_DISTANCE=false` → `insufficient_data`.
5. **Missing/short energy data** (< 6 months) → solar `insufficient_data`; 6–11 months → `ANNUALISED_PARTIAL` with ±15%; Green Home with < 12 months → `INSUFFICIENT_DATA`.
6. **Conflicting signals** (e.g. `home_ownership=RENTER` but high home-improvement spend) → rules win: `NOT_ELIGIBLE` for Green Home/Solar.
7. **Existing solar** → `NOT_ELIGIBLE`; **missing** `has_solar` → `INSUFFICIENT_DATA`.
8. **Flat** → solar `NOT_ELIGIBLE`.
9. **Already paperless** → `NOT_ELIGIBLE`; `requires_paper_comms=true` → `NOT_ELIGIBLE`.
10. **Vulnerability flag** → `ELIGIBLE_WITH_CONDITIONS` + `EL_COND_HUMAN_REVIEW`; gate marks "no automated message".
11. **Opted out of marketing** → `NOT_ELIGIBLE` for every product.
12. **Stale data (> 90 days)** → `INSUFFICIENT_DATA`.
13. **Zero distance / zero statements** → valid zero result.
14. **Negative reduction** (force EV kWh/km×grid > ICE factor) → signed value kept, `claim_reduction=false`, `GATE_NO_REDUCTION`.
15. **Units:** miles vs km, Wh vs kWh converted correctly; unknown unit → `calculation_error`.
16. **Factor provenance:** `DEV_ILLUSTRATIVE` factor → `PROVISIONAL`; missing source/unit/geography → loader rejects; wrong geography → `not_applicable`; EV baseline/scenario boundary mismatch → `calculation_error`.
17. **Reproducibility:** same inputs twice → identical JSON (ignoring `calculated_at`).
18. **Rule precedence:** a customer with both a HARD_BLOCK fail and missing data → `NOT_ELIGIBLE`.
19. **Priority:** within-product normalisation; paper priority never compared with CO2e; individual

components always returned.

20. **API:** malformed body and unsupported product → 422, no fabricated result.
21. **Interface:** Opportunity output validates against the `Opportunity` pydantic schema and passes through `propensity/confidence/reason_codes/model_version` unchanged.
22. **Probability range:** propensity or confidence outside [0, 1] (or non-numeric) → 422 `INVALID_ML_PAYLOAD`.
23. **Reason-code registry:** a reason code not in `reason_codes.yaml` → rejected; known codes pass through unchanged.
24. **Cutoff/leakage:** an energy month or transaction dated after `cutoff_date` → `calculation_error` `INPUT_AFTER_CUTOFF`.
25. **Estimate labelling:** every `estimated` impact has `is_estimate=true` and the label; no emitted string contains "saved" / "will save".
26. **Confidence independence:** changing `confidence` alone never changes `eligibility_status`.

10.3 Quality gates

- ⩾ 90% line coverage on `carbon_engine/` and `eligibility/`.
- One run command: `pytest -q`; pin dependencies in `requirements.txt`.
- No test may call the network or an LLM.

---

11. Documentation to generate

`docs/carbon_methodology.md` must contain, per product: purpose/scope, formula, baseline vs scenario, time horizon, units, boundary, every factor ID with source/year/version, every assumption with source, conditions that cause `insufficient_data`, and the line **"Outputs are estimates of potential impact, not measured outcomes."** Also document: the cutoff date and outcome window used, the Green Home vs Solar scope boundary, the rule thresholds in 4.4 as synthetic authorised criteria, and that final approval stays with the institution's normal processes.

---

12. Build order (do not skip ahead)

1. `schemas.py`, `units.py`, `ranges.py`, `factors.py` + tests
2. **EV end-to-end first:** `ev.py` + eligibility rules G*, L*, EV* + tests + orchestrator + one Opportunity JSON
3. Stop and show output; confirm schema with teammates
4. Solar, Green Home, Paperless (same pattern)
5. Gate + priority
6. API wrapper + mock mode
7. Methodology doc

Never cut: the eligibility gate, documented formulas with ranges, unit tests, insufficient-data handling.

---

13. Open items for the human owner (do NOT let the agent guess these)

- [ ] Replace all `DEV_ILLUSTRATIVE` / `AGGREGATOR_UNVERIFIED` factors with values from the official DESNZ 2026 flat file; confirm boundaries (EV charging factor vs per-km vehicle factor must match).
- [ ] Cite a source for every row in `assumptions.csv`.
- [ ] Confirm field names and units with Person 1 (data) and the Opportunity JSON with Persons 2, 4, 5.
- [ ] Confirm Green Home scope (fabric + heating only) and Solar scope with the team.
- [ ] Confirm whether impact is computed for all customers or only shortlisted ones (affects the P95

normalisation pool).

- [ ] Decide whether Paperless CO2e stays disabled (recommended).
- [ ] Confirm the outcome window (default 90 days) and cutoff date with Persons 1 and 2, and the confidence method with Person 2.